

memtorch.bh.memristor

Submodule containing various behavioral memristor models, that extend

`memtorch.bh.memristor.Memristor`.

memtorch.bh.memristor.Memristor

Base class used to model memristive device behavior.

```
class memtorch.bh.memristor.Memristor.Memristor(r_off, r_on, time_series_resolution, pos_write_threshold=0, neg_write_threshold=0) \[source\]
```

Bases: `abc.ABC`

- Parameters:
- `r_off` (*float*) – Off (maximum) resistance of the device (ohms).
 - `r_on` (*float*) – On (minimum) resistance of the device (ohms).
 - `time_series_resolution` (*float*) – Time series resolution (s).
 - `pos_write_threshold` (*float*) – Positive write threshold voltage (V).
 - `neg_write_threshold` (*float*) – Negative write threshold voltage (V).

`get_resistance()` [\[source\]](#)

Method to determine the resistance of a memristive device.

Returns: The devices resistance (ohms).

Return type: *float*

`plot_bipolar_switching_behaviour(memristor, voltage_signal_amplitude, voltage_signal_frequency, log_scale=True, return_result=False)` [\[source\]](#)

Method to plot the DC bipolar switching behaviour of a given device.

- Parameters:
- `memristor` (*memtorch.bh.memristor.Memristor.Memristor*) – Memristor.
 - `voltage_signal_amplitude` (*float*) – Voltage signal amplitude (V).
 - `voltage_signal_frequency` (*float*) – Voltage signal frequency (Hz).
 - `log_scale` (*bool*) – Plot the y-axis (current) using a sy  [latest](#)  scale (True).
 - `return_result` (*bool*) – Voltage and current signals are returned

(True).

plot_hysteresis_loop(memristor, duration, voltage_signal_amplitude, voltage_signal_frequency, log_scale=False, return_result=False) [\[source\]](#)

Method to plot the hysteresis loop of a given device.

- Parameters:
- **memristor** ([memtorch.bh.memristor.Memristor.Memristor](#)) – Memristor.
 - **duration** ([float](#)) – Duration (s).
 - **voltage_signal_amplitude** ([float](#)) – Voltage signal amplitude (V).
 - **voltage_signal_frequency** ([float](#)) – Voltage signal frequency (Hz)
 - **log_scale** ([bool](#)) – Plot the y-axis (current) using a symmetrical log scale (True).
 - **return_result** ([bool](#)) – Voltage and current signals are returned (True).

set_conductance(conductance) [\[source\]](#)

Method to manually set the conductance of a memristive device.

- Parameters: **conductance** ([float](#)) – Conductance to set.

simulate(voltage_signal) [\[source\]](#)

Method to determine the equivalent conductance of a memristive device when a given voltage signal is applied.

- Parameters: **voltage_signal** ([torch.Tensor](#)) – A discrete voltage signal with resolution time_series_resolution.

- Returns: A tensor containing the equivalent device conductance for each simulated timestep.

- Return type: [torch.Tensor](#)

memtorch.bh.memristor.Memristor.plot_bipolar_switching_behaviour(memristor_model, voltage_signal_amplitude, voltage_signal_frequency, log_scale=True, return_result=False) [\[source\]](#)

Method to plot the DC bipolar switching behaviour of a given device.

- Parameters:
- **memristor_model** ([memtorch.bh.memristor.Memristor.Memristor](#)) – Memristor model.
 - **voltage_signal_amplitude** ([float](#)) – Voltage signal amplitude (V).
 - **voltage_signal_frequency** ([float](#)) – Voltage signal frequency (Hz)

- **log_scale** (*bool*) – Plot the y-axis (current) using a symmetrical log scale (True).
- **return_result** (*bool*) – Voltage and current signals are returned (True).

memtorch.bh.memristor.Memristor.plot_hysteresis_loop(*memristor_model, duration, voltage_signal_amplitude, voltage_signal_frequency, log_scale=False, return_result=False*)
[\[source\]](#)

Method to plot the hysteresis loop of a given device.

- Parameters:**
- **memristor_model** (*memtorch.bh.memristor.Memristor.Memristor*) – Memristor model.
 - **duration** (*float*) – Duration (s).
 - **voltage_signal_amplitude** (*float*) – Voltage signal amplitude (V).
 - **voltage_signal_frequency** (*float*) – Voltage signal frequency (Hz)
 - **log_scale** (*bool*) – Plot the y-axis (current) using a symmetrical log scale (True).
 - **return_result** (*bool*) – Voltage and current signals are returned (True).

Returns: Voltage and current signals.

Return type: *tuple*

Currently supported memristor models are listed below:

memtorch.bh.memristor.LinearIonDrift

class

memtorch.bh.memristor.LinearIonDrift.LinearIonDrift(*time_series_resolution=0.0001, u_v=1e-14, d=1e-08, r_on=100, r_off=16000.0, pos_write_threshold=0.55, neg_write_threshold=-0.55, p=1, **kwargs*) [\[source\]](#)

Bases: `memtorch.bh.memristor.Memristor.Memristor`

Linear Ion behavioural drift model.

- Parameters:**
- **time_series_resolution** (*float*) – Time series resolution (s).
 - **u_v** (*float*) – Dopant drift mobility of the device material.
 - **d** (*float*) – Device length (m).
 - **r_on** (*float*) – On (minimum) resistance of the device (ohms).
 - **r_off** (*float*) – Off (maximum) resistance of the device (ohms).
 - **pos_write_threshold** (*float*) – Positive write threshold voltage (V).
 - **neg_write_threshold** (*float*) – Negative write threshold voltage (V).
 - **p** (*int*) – Joglekar window p constant.

current(voltage) [\[source\]](#)

Method to determine the current of the model given an applied voltage.

Parameters: **voltage** (*float*) – The current applied voltage (V).

Returns: The observed current (A).

Return type: *float*

dxdt(current) [\[source\]](#)

Method to determine the derivative of the state variable, dx/dt .

Parameters: **current** (*float*) – The observed current (A).

Returns: The derivative of the state variable, dx/dt .

Return type: *float*

plot_bipolar_switching_behaviour(voltage_signal_amplitude=5, voltage_signal_frequency=2.5, log_scale=True, return_result=False) [\[source\]](#)

Method to plot the DC bipolar switching behaviour of a given device.

Parameters:

- **memristor** ([memtorch.bh.memristor.Memristor.Memristor](#)) – Memristor.
- **voltage_signal_amplitude** (*float*) – Voltage signal amplitude (V).
- **voltage_signal_frequency** (*float*) – Voltage signal frequency (Hz)
- **log_scale** (*bool*) – Plot the y-axis (current) using a symmetrical log scale (True).
- **return_result** (*bool*) – Voltage and current signals are returned (True).

plot_hysteresis_loop(duration=4, voltage_signal_amplitude=5, voltage_signal_frequency=2.5, return_result=False) [\[source\]](#)

Method to plot the hysteresis loop of a given device.

Parameters:

- **memristor** ([memtorch.bh.memristor.Memristor.Memristor](#)) – Memristor.
- **duration** (*float*) – Duration (s).
- **voltage_signal_amplitude** (*float*) – Voltage signal amplitude (V).
- **voltage_signal_frequency** (*float*) – Voltage signal frequency (Hz)
- **log_scale** (*bool*) – Plot the y-axis (current) using a symmetrical log scale (True).

- **return_result** (*bool*) – Voltage and current signals are returned (True).

set_conductance(*conductance*) [\[source\]](#)

Method to manually set the conductance of a memristive device.

Parameters: **conductance** (*float*) – Conductance to set.

simulate(*voltage_signal*, *return_current=False*) [\[source\]](#)

Method to determine the equivalent conductance of a memristive device when a given voltage signal is applied.

Parameters: **voltage_signal** (*torch.Tensor*) – A discrete voltage signal with resolution *time_series_resolution*.

Returns: A tensor containing the equivalent device conductance for each simulated timestep.

Return type: *torch.Tensor*

memtorch.bh.memristor.VTEAM

```
class memtorch.bh.memristor.VTEAM.VTEAM(time_series_resolution=1e-10, r_off=1000, r_on=50, d=3e-09, k_on=-10, k_off=0.0005, alpha_on=3, alpha_off=1, v_on=-0.2, v_off=0.02, x_on=0, x_off=3e-09, **kwargs) \[source\]
```

Bases: `memtorch.bh.memristor.Memristor.Memristor`

VTEAM memristor model (<https://asic2.group/tools/memristor-models/>).

- Parameters:
- **time_series_resolution** (*float*) – Time series resolution (s).
 - **r_off** (*float*) – Off (maximum) resistance of the device (ohms).
 - **r_on** (*float*) – On (minimum) resistance of the device (ohms).
 - **d** (*float*) – Device length (m).
 - **k_on** (*float*) – *k_{on}* model parameter.
 - **k_off** (*float*) – *k_{off}* model parameter.
 - **alpha_on** (*float*) – *alpha_{on}* model parameter.
 - **alpha_off** (*float*) – *alpha_{off}* model parameter.
 - **v_on** (*float*) – Positive write threshold voltage (V).
 - **v_off** (*float*) – Negative write threshold voltage (V).
 - **x_on** (*float*) – *x_{on}* model parameter.
 - **x_off** (*float*) – *x_{off}* model parameter.

current(voltage) [\[source\]](#)

Method to determine the current of the model given an applied voltage.

Parameters: **voltage** (*float*) – The current applied voltage (V).

Returns: The observed current (A).

Return type: *float*

dxdt(voltage) [\[source\]](#)

Method to determine the derivative of the state variable.

Parameters: **voltage** (*float*) – The current applied voltage (V).

Returns: The derivative of the state variable.

Return type: *float*

plot_bipolar_switching_behaviour(voltage_signal_amplitude=1.5, voltage_signal_frequency=50000000.0, log_scale=True, return_result=False) [\[source\]](#)

Method to plot the DC bipolar switching behaviour of a given device.

Parameters:

- **memristor** (*[memtorch.bh.memristor.Memristor.Memristor](#)*) – Memristor.
- **voltage_signal_amplitude** (*float*) – Voltage signal amplitude (V).
- **voltage_signal_frequency** (*float*) – Voltage signal frequency (Hz)
- **log_scale** (*bool*) – Plot the y-axis (current) using a symmetrical log scale (True).
- **return_result** (*bool*) – Voltage and current signals are returned (True).

plot_hysteresis_loop(duration=2e-07, voltage_signal_amplitude=1, voltage_signal_frequency=50000000.0, return_result=False) [\[source\]](#)

Method to plot the hysteresis loop of a given device.

Parameters:

- **memristor** (*[memtorch.bh.memristor.Memristor.Memristor](#)*) – Memristor.
- **duration** (*float*) – Duration (s).
- **voltage_signal_amplitude** (*float*) – Voltage signal amplitude (V).
- **voltage_signal_frequency** (*float*) – Voltage signal frequency (Hz)
- **log_scale** (*bool*) – Plot the y-axis (current) using a symmetrical log scale (True).
- **return_result** (*bool*) – Voltage and current signals are returned

(True).

set_conductance(conductance) [\[source\]](#)

Method to manually set the conductance of a memristive device.

Parameters: **conductance** (*float*) – Conductance to set.

simulate(voltage_signal, return_current=False) [\[source\]](#)

Method to determine the equivalent conductance of a memristive device when a given voltage signal is applied.

Parameters: **voltage_signal** (*torch.Tensor*) – A discrete voltage signal with resolution *time_series_resolution*.

Returns: A tensor containing the equivalent device conductance for each simulated timestep.

Return type: *torch.Tensor*

memtorch.bh.memristor.Data_Driven

```
class memtorch.bh.memristor.Data_Driven.Data_Driven(time_series_resolution=1e-08,
r_off=17000.0, r_on=1280, A_p=743.47, A_n=-68012.28374, t_p=6.51, t_n=0.31645,
k_p=0.000511, k_n=0.00117, r_p=[16719, 0], r_n=[29304.82557, 23692.77225], eta=1, a_p=0.24,
a_n=0.24, b_p=3, b_n=3, **kwargs) \[source\]
```

Bases: `memtorch.bh.memristor.Memristor.Memristor`

A Data-Driven Verilog-A ReRAM Model (<https://eprints.soton.ac.uk/411693/>).

Parameters:

- **time_series_resolution** (*float*) – Time series resolution (s).
- **r_off** (*float*) – Off (maximum) resistance of the device (ohms).
- **r_on** (*float*) – On (minimum) resistance of the device (ohms).
- **A_p** (*float*) – A_p model parameter.
- **A_n** (*float*) – A_n model parameter.
- **t_p** (*float*) – t_p model parameter.
- **t_n** (*float*) – t_n model parameter.
- **k_p** (*float*) – k_p model parameter.
- **k_n** (*float*) – k_n model parameter.
- **r_p** (*float*) – r_p voltage-dependent resistive boundary function coefficients.
- **r_n** (*float*) – r_n voltage-dependent resistive boundary function coefficients.
- **eta** (*int*) – Switching direction to stimulus polarity.

- **a_p** (*float*) – a_p model parameter.
- **a_n** (*float*) – a_n model parameter.
- **b_p** (*float*) – b_p model parameter.
- **b_n** (*float*) – b_n model parameter.

current(*voltage*) [\[source\]](#)

Method to determine the current of the model given an applied voltage.

Parameters: **voltage** (*float*) – The current applied voltage (V).

Returns: The observed current (A).

Return type: *float*

plot_bipolar_switching_behaviour(*voltage_signal_amplitude=1.5, voltage_signal_frequency=10000.0, log_scale=True, return_result=False*) [\[source\]](#)

Method to plot the DC bipolar switching behaviour of a given device.

Parameters:

- **memristor** (*memtorch.bh.memristor.Memristor.Memristor*) – Memristor.
- **voltage_signal_amplitude** (*float*) – Voltage signal amplitude (V).
- **voltage_signal_frequency** (*float*) – Voltage signal frequency (Hz)
- **log_scale** (*bool*) – Plot the y-axis (current) using a symmetrical log scale (True).
- **return_result** (*bool*) – Voltage and current signals are returned (True).

plot_hysteresis_loop(*duration=0.001, voltage_signal_amplitude=1.5, voltage_signal_frequency=10000.0, return_result=False*) [\[source\]](#)

Method to plot the hysteresis loop of a given device.

Parameters:

- **memristor** (*memtorch.bh.memristor.Memristor.Memristor*) – Memristor.
- **duration** (*float*) – Duration (s).
- **voltage_signal_amplitude** (*float*) – Voltage signal amplitude (V).
- **voltage_signal_frequency** (*float*) – Voltage signal frequency (Hz)
- **log_scale** (*bool*) – Plot the y-axis (current) using a symmetrical log scale (True).
- **return_result** (*bool*) – Voltage and current signals are returned (True).

resistance(*voltage*) [\[source\]](#)

Method to determine the resistance of the model given an applied voltage.

Parameters: **voltage** (*float*) – The current applied voltage (V).

Returns: The observed resistance (Ω).

Return type: *float*

set_conductance(*conductance*) [\[source\]](#)

Method to manually set the conductance of a memristive device.

Parameters: **conductance** (*float*) – Conductance to set.

simulate(*voltage_signal*, *return_current=False*) [\[source\]](#)

Method to determine the equivalent conductance of a memristive device when a given voltage signal is applied.

Parameters: **voltage_signal** (*torch.Tensor*) – A discrete voltage signal with resolution *time_series_resolution*.

Returns: A tensor containing the equivalent device conductance for each simulated timestep.

Return type: *torch.Tensor*

memtorch.bh.memristor.Data_Driven2021

class

memtorch.bh.memristor.Data_Driven2021.Data_Driven2021(*time_series_resolution=1e-10*, *r_off=3000*, *r_on=1600*, *A_p=600.10075*, *A_n=-34.5988399*, *t_p=-0.0212028*, *t_n=-0.05343997*, *k_p=0.000511*, *k_n=0.00117*, *r_p=[2699.2336, -672.930205]*, *r_n=[649.413746, -1474.32358]*, *a_p=0.32046175*, *a_n=0.32046175*, *b_p=2.71689828*, *b_n=2.71689828*, ****kwargs**) [\[source\]](#)

Bases: `memtorch.bh.memristor.Memristor.Memristor`

An updated Data-Driven Verilog-A ReRAM Model. Based on the model used in the article at the following address: <https://arxiv.org/abs/2012.02267>. The main difference with the 2018 version (simply Data_Driven.py) is that it is less computationally demanding and achieves similar results. The creator of the data driven model as decided to return to the 2017 version for this reason in his recent works. The default parameters were determined experimentally.

Parameters: • **time_series_resolution** (*float*) – Time series resolution (s).

- **r_off** (*float*) – Off (maximum) resistance of the device (ohms).
- **r_on** (*float*) – On (minimum) resistance of the device (ohms).
- **A_p** (*float*) – A_p model parameter.
- **A_n** (*float*) – A_n model parameter.
- **t_p** (*float*) – t_p model parameter.
- **t_n** (*float*) – t_n model parameter.
- **k_p** (*float*) – k_p model parameter.
- **k_n** (*float*) – k_n model parameter.
- **r_p** (*float*) – r_p voltage-dependent resistive boundary function coefficients.
- **r_n** (*float*) – r_n voltage-dependent resistive boundary function coefficients.
- **a_p** (*float*) – a_p model parameter.
- **a_n** (*float*) – a_n model parameter.
- **b_p** (*float*) – b_p model parameter.
- **b_n** (*float*) – b_n model parameter.

current(voltage) [\[source\]](#)

Method to determine the current of the model given an applied voltage.

Parameters: **voltage** (*float*) – The current applied voltage (V).

Returns: The observed current (A).

Return type: *float*

dRdt(voltage) [\[source\]](#)

Function to return dR/dT From the 2017/2021 paper model calculations

Parameters:

- **voltage** (*float*) –
- **current applied voltage (V)** (*The*) –
- **a1** (*a0,*) –
- **value of a0 or a1** (*The*) –

Returns:

- *float*
- *The derivative with respect to time of the resistance*

plot_bipolar_switching_behaviour(voltage_signal_amplitude=1.5, voltage_signal_frequency=10000.0, log_scale=True, return_result=False) [\[source\]](#)

Method to plot the DC bipolar switching behaviour of a given device.

- Parameters:
- **memristor** ([memtorch.bh.memristor.Memristor.Memristor](#)) – Memristor.
 - **voltage_signal_amplitude** ([float](#)) – Voltage signal amplitude (V).
 - **voltage_signal_frequency** ([float](#)) – Voltage signal frequency (Hz)
 - **log_scale** ([bool](#)) – Plot the y-axis (current) using a symmetrical log scale (True).
 - **return_result** ([bool](#)) – Voltage and current signals are returned (True).

plot_hysteresis_loop(duration=0.001, voltage_signal_amplitude=1.5, voltage_signal_frequency=10000.0, return_result=False) [\[source\]](#)

Method to plot the hysteresis loop of a given device.

- Parameters:
- **memristor** ([memtorch.bh.memristor.Memristor.Memristor](#)) – Memristor.
 - **duration** ([float](#)) – Duration (s).
 - **voltage_signal_amplitude** ([float](#)) – Voltage signal amplitude (V).
 - **voltage_signal_frequency** ([float](#)) – Voltage signal frequency (Hz)
 - **log_scale** ([bool](#)) – Plot the y-axis (current) using a symmetrical log scale (True).
 - **return_result** ([bool](#)) – Voltage and current signals are returned (True).

r_pn(voltage, a0, a1) [\[source\]](#)

Function to return $r_p(v)$ or $r_n(v)$ From the 2017/2021 paper model calculations

- Parameters:
- **voltage** ([float](#)) – The current applied voltage (V).
 - **a1** (a0,) – The value of a0 and a1

- Returns:
- [float](#)
 - The r_p or r_n resistance (Ω).

resistance(voltage) [\[source\]](#)

Method to determine the resistance of the model given an applied voltage. Using the 2017/2021 model

- Parameters:
- **voltage** ([float](#)) – The current applied voltage (V).

- Returns:
- The observed resistance (Ω).

Return type: `float`

s_pn(voltage, A, t) [\[source\]](#)

Function to return $sp(v)$ or $sn(v)$ From the 2017/2021 paper model calculations

Parameters:

- voltage (`float`) –
- current applied voltage (V) (The) –
- t (A,) –
- value of model params A (A_p or A_n) or t(t_p or t_n) (The) –

Returns:

- `float`
- The sp or sn variability.

set_conductance(conductance) [\[source\]](#)

Method to manually set the conductance of a memristive device.

Parameters: conductance (`float`) – Conductance to set.

simulate(voltage_signal, return_current=False) [\[source\]](#)

Method to determine the equivalent conductance of a memristive device when a given voltage signal is applied.

Parameters: voltage_signal (`torch.Tensor`) – A discrete voltage signal with resolution time_series_resolution.

Returns: A tensor containing the equivalent device conductance for each simulated timestep.

Return type: `torch.Tensor`

memtorch.bh.memristor.Stanford_PKU

class

```
memtorch.bh.memristor.Stanford_PKU.Stanford_PKU(time_series_resolution=0.0001,
r_off=218586, r_on=542, gap_init=2e-10, g_0=2.5e-10, V_0=0.25, I_0=0.001, read_voltage=0.1,
T_init=298, R_th=2100.0, gamma_init=16, beta=0.8, t_ox=1.2e-08, F_min=1400000000.0,
vel_0=10, E_a=0.6, a_0=2.5e-10, delta_g_init=0.02, model_switch=0, T_crit=450, T_smth=500,
**kwargs) \[source\]
```

Bases: `memtorch.bh.memristor.Memristor.Memristor`

Stanford PKU memristor model (<https://nano.stanford.edu/stanford-rram-model>).

- Parameters:**
- **time_series_resolution** (*float*) – Time series resolution (s).
 - **r_off** (*float*) – Off (maximum) resistance of the device (ohms).
 - **r_on** (*float*) – On (minimum) resistance of the device (ohms).
 - **gap_init** (*float*) – Initial gap distance (m).
 - **g_0** (*float*) – g_0 model parameter.
 - **V_0** (*float*) – V_0 model parameter.
 - **I_0** (*float*) – I_0 model parameter.
 - **read_voltage** (*float*) – Read voltage (V) to determine the device's conductance.
 - **T_init** (*float*) – Initial room temperature.
 - **R_th** (*float*) – Thermal resistance.
 - **gamma_init** (*float*) – gamma_init model parameter.
 - **beta** (*float*) – beta model parameter.
 - **t_ox** (*float*) – Oxide thickness (m).
 - **F_min** (*float*) – Minimum field requirement to enhance gap formation.
 - **vel_0** (*float*) – vel_0 model parameter.
 - **E_a** (*float*) – Activation energy.
 - **a_0** (*float*) – Atom spacing.
 - **delta_g_init** (*float*) – Initial delta_g value.
 - **model_switch** (*int*) – Switch to select standard model (0) or dynamic model (1).
 - **T_crit** (*float*) – Threshold temperature (K) for significant random variations.
 - **T_smth** (*float*) – Activation energy for vacancy generation.

T_current(*voltage*, *current*) [\[source\]](#)

Method to determine the thermal current of the model given an applied voltage and current.

- Parameters:**
- **voltage** (*float*) – The current applied voltage (V).
 - **current** (*float*) – The current applied current (A).

Returns: The observed thermal current (A).

Return type: *float*

current(*voltage*) [\[source\]](#)

Method to determine the current of the model given an applied voltage.

- Parameters:**
- **voltage** (*float*) – The current applied voltage (V).

Returns: The observed current (A).

Return type: `float`

dg_dt(*voltage*, *current*) [\[source\]](#)

Method to determine the derivative of the gap length.

Parameters:

- **voltage** (`float`) – The current applied voltage (V).
- **current** (`float`) – The current applied current (A).

Returns: The derivative of the gap length.

Return type: `float`

plot_bipolar_switching_behaviour(*voltage_signal_amplitude*=1.5, *voltage_signal_frequency*=0.05, *log_scale*=True, *return_result*=False) [\[source\]](#)

Method to plot the DC bipolar switching behaviour of a given device.

Parameters:

- **memristor** ([memtorch.bh.memristor.Memristor.Memristor](#)) – Memristor.
- **voltage_signal_amplitude** (`float`) – Voltage signal amplitude (V).
- **voltage_signal_frequency** (`float`) – Voltage signal frequency (Hz)
- **log_scale** (`bool`) – Plot the y-axis (current) using a symmetrical log scale (True).
- **return_result** (`bool`) – Voltage and current signals are returned (True).

plot_hysteresis_loop(*duration*=0.5, *voltage_signal_amplitude*=1.5, *voltage_signal_frequency*=10, *log_scale*=False, *return_result*=False) [\[source\]](#)

Method to plot the hysteresis loop of a given device.

Parameters:

- **memristor** ([memtorch.bh.memristor.Memristor.Memristor](#)) – Memristor.
- **duration** (`float`) – Duration (s).
- **voltage_signal_amplitude** (`float`) – Voltage signal amplitude (V).
- **voltage_signal_frequency** (`float`) – Voltage signal frequency (Hz)
- **log_scale** (`bool`) – Plot the y-axis (current) using a symmetrical log scale (True).
- **return_result** (`bool`) – Voltage and current signals are returned (True).

set_conductance(conductance) [\[source\]](#)

Method to manually set the conductance of a memristive device.

Parameters: conductance ([float](#)) – Conductance to set.

simulate(voltage_signal, return_current=False) [\[source\]](#)

Method to determine the equivalent conductance of a memristive device when a given voltage signal is applied.

Parameters: voltage_signal ([torch.Tensor](#)) – A discrete voltage signal with resolution time_series_resolution.

Returns: A tensor containing the equivalent device conductance for each simulated timestep.

Return type: [torch.Tensor](#)